TRECAP: UVM and Formal Verification

Reusable simulation and local property proofs for a finite FPGA audio pipeline

TRECAP project

October 17, 2026

Abstract

This supplement adds transaction-based UVM simulation and local formal verification to the registered TSH2 finite-file design. It separates three kinds of evidence: numerical comparisons on executed traces, inductive properties under explicit assumptions, and reachable witness traces. Existing RTL, reference fixtures and physical results remain identifiable independently of the new verification environment. The result is a bounded engineering case study rather than a claim of complete chip sign-off.

Table 1: Registered supplemental result. Counts refer to distinct evidence types.

Evidence	Accepted result
UVM simulation	Three seeds, 12 completed epochs, three interrupted prefixes and three reset recoveries included in the 12 completed epochs.
Exact numerical checks	18,432 reconstructed samples, 13,932 canonical-bin rows and 94,440 packet bytes in completed epochs.
Formal proof	16 safety/equivalence jobs discharged for the stated instances and assumptions.
Formal reachability	14 bounded-cover jobs, reported separately from proofs.
Deliberate failures	One UVM wrong-reference test and three false-property formal tests detected the intended violation.

No production RTL changed. The simulator campaign required one explicit bounded recovery from a startup crash. The original failed aggregate remains evidence of that failure, rather than being overwritten by the accepted recovery manifest.

1 Scope and verification contract

The verification target is the fixed-point FFT, threshold, IFFT/WOLA and spectral-export pipeline used in the registered TSH2 profile. The UVM environment reuses the admitted short synthetic corpus and exact reference outputs. Local formal harnesses instantiate unchanged implementation modules. The historical campaign of 14 RTL suites and 58 physical epochs is separate evidence; it was not retroactively executed using UVM.

The acceptance policy was recorded before results: genuine UVM execution, passive transfer observation, exact numerical scoreboards, nonzero observed event counts, bounded runner completion, actual formal-engine results, explicit assumptions, cover witnesses and deliberate checker failures. Failed infrastructure or harness-development attempts remain distinct from successful verification runs.

Table 2: Evidence boundaries and the questions they answer.

Method	Question	Limit
UVM simulation	Does the DUT satisfy the checked contracts for executed transactions?	Finite vectors and schedules, not all possible traces.
Formal safety proof	Can the named property fail in the instantiated transition system under the assumptions?	The exact module, parameters, reset model and assumptions.
Formal cover	Can a specified scenario occur within the search depth?	A witness establishes reachability, not universal correctness.
Negative control	Does the verification flow detect a deliberate violation?	Qualifies a specified checker path, not every possible defect.
Physical campaign	Did the registered implementation reproduce expected captures on DE1-SoC?	Historical board conditions; not rerun for this supplement.

The original 46 broad requirements retain their historical 30 partial and 16 excluded dispositions. This supplement supplies additional evidence for selected numerical, framing, flow-control and reset contracts. A framework label does not automatically close a requirement.

2 UVM environment and observation model

The environment uses the actual Accellera UVM 1.2 source library bundled with ModelSim Intel FPGA Starter 2020.1. A preliminary library probe executed factory construction, test phases and objections before the DUT environment was admitted. The completed DUT regression, rather than that probe, supplies the numerical evidence.

An `epoch_item` carries case identity and the reset-abort selection. The sequence submits one interrupted epoch followed by four complete epochs through a UVM sequencer and driver. The driver controls PCM presentation, start/reset and independent readiness of the waveform and packet consumers through a virtual interface. Stimulus is scheduled away from the sampling edge. The passive monitor observes accepted transfers at the rising edge and sends separate analysis transactions to the scoreboard.

The scoreboard compares each accepted PCM token, padded source token, waveform output, canonical-bin record and exported packet byte with immutable admitted fixtures. It checks record identity and ordering as well as values. Final time-domain metrics are calculated independently by absolute output index. A second Python checker reads the recorded observations and reconciles them with the fixtures after the simulator exits.

The fixed corpus contains 1,024 input samples per completed epoch, 1,536 full-tail output samples, nine frames and 1,161 canonical-bin records. Thresholds are zero and 10^8; each is exercised in dense and sparse packet mode. Three explicit seeds drive distinct reproducible stall schedules. For each seed, the first completed case is also the exact recovery check after the interrupted prefix. Recovery epochs must not be added a second time to the completed-epoch denominator.

Tool capability boundary. The Starter simulator reports restrictions on advanced verification features. This environment uses deterministic seeded UVM sequences and observed event counters. It does not claim solver-constrained randomization, executed covergroups, simulator code-coverage closure or a UVM-aware debugger. The library is compiled with `UVM_NO_DPI`; the disabled DPI-dependent facilities are not part of the acceptance contract. UVM 1.2 is identified explicitly, without a claim of IEEE 1800.2-2020 library conformance.

2.1 Executed matrix and coverage interpretation

The registered seeds are 1, 7 and 20261017. Each completes the same four numerical cases with a distinct readiness and input-gap schedule. All completed output values and final metrics match their case-specific fixtures. FIFO empty, one-entry and full states, input gaps, waveform stalls, packet stalls and final-output hold are observed in the required completed cases. Masked-bin coverage applies to thresholded cases rather than threshold zero.

Table 3: UVM numerical work counted only in completed epochs.

Observation	Per completed epoch	All 12 epochs
Accepted PCM input	1,024	12,288
Padded source and reconstructed output, each	1,536	18,432
Canonical-bin records	1,161	13,932
Frames	9	108

Packet length depends on the threshold and encoding. The total 94,440 bytes is checked byte for

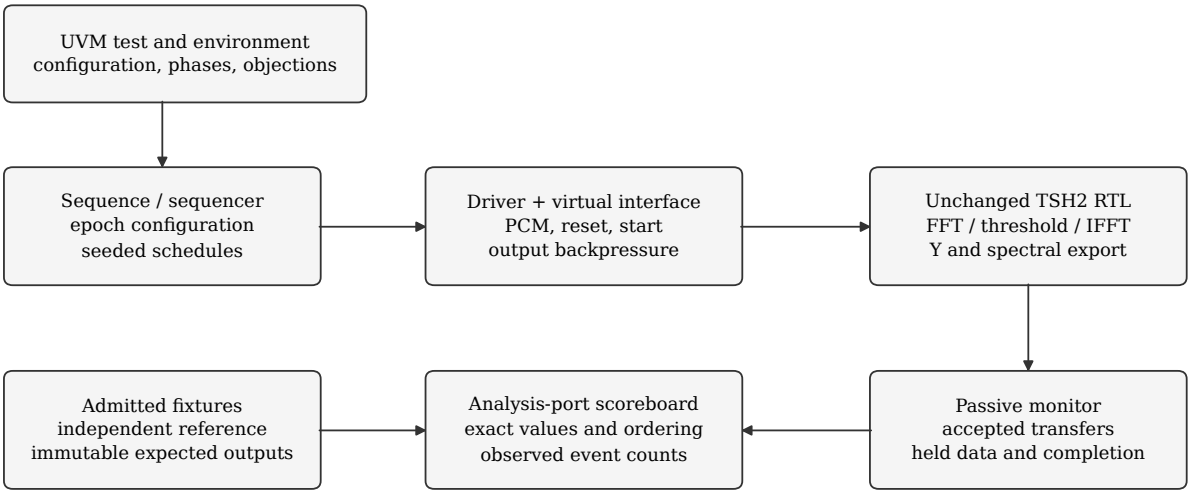


Figure 1: UVM verification architecture. Diagrammatic roles are separated from measured results.

byte, not inferred from a duration. The three interrupted prefixes are checked until reset, then retired as aborted transactions. Their partial counts are retained separately and excluded from this table.

The final waveform token is held for 33, 39 and 57 sampled clock edges for seeds 1, 7 and 20261017 respectively. The checker requires held data/index stability, continued busy ownership and no premature completion. Completion is accepted only after the final transfer and drain conditions. Event counters establish that these scenarios occurred; they are not a simulator code-coverage percentage or an exhaustive exploration of schedules.

The accepted recovery manifest binds every selected run to its actual compiled DUT, fixture and harness copies, compile receipts, UVM library identity and executable hashes. It aggregates successful individual executions after validating their provenance. It does not relabel the failed original aggregate as a success.

3 Formal harnesses, assumptions and proof results

Formal jobs use a pinned official OSS CAD Suite release. Each job binds unchanged implementation sources, a property harness, parameter values, the frontend, solver/engine, search depth and a deadline. An accepted safety result must come from the actual engine. A successful package-integrity check does not execute a proof.

The harnesses use immediate SystemVerilog assertions and covers with sampled history. These are formal properties analyzed by a solver; they are not advertised as a concurrent-SVA test suite. Sequential proofs assume an initial sampled reset. Subsequent input controls remain unconstrained except where a case explicitly states otherwise. Asynchronous-reset normalization is interpreted at the formal sampling boundary and does not establish electrical reset behavior or metastability safety.

3.1 Synchronous FIFO

The accepted operations are determined from the pre-edge state:

$$p = \text{push_i} \wedge \neg\text{full}, \qquad q = \text{pop_i} \wedge \neg\text{empty}.$$

Outside reset/clear, occupancy must update as $n' = n + p - q$ and remain within capacity. At full occupancy, simultaneous requests accept only the pop; at empty occupancy, they accept only the push. The output is registered on an accepted pop. Uninitialized RAM is not assumed to contain valid data.

The deployed adapter instantiates the generic `stream_fifo` at 32-bit width and depth 1,024 for its Y and spectral-output queues. Its PCM input queue is a separate implementation in `trecap_pcm_stream_source`. A generic FIFO proof at depth 256 therefore does not prove the PCM queue. Reduced-width/depth ordering checks and production-size control checks are reported separately in the result ledger.

3.2 Rounded shift and saturation

The `trecap_round_sat` harness uses an independently expressed signed floor-quotient and nonnegative-remainder oracle. It checks the rounded value, signed clipping and the high/low/any saturation flags for every input bit pattern of each declared instance. The matrix includes the selected 29-to-28-bit shift-one setting and large-shift boundaries. Passing selected parameter instances does not prove all possible parameter combinations.

3.3 Threshold-mask protocol and decision rules

The mask harness instantiates the default 28-bit complex-input and 56-bit squared-magnitude implementation. Explicit top-level inputs preserve shared unconstrained controls and payloads through the SystemVerilog frontend. A shadow acceptance record tracks the accepted threshold and metadata; it is independent of the DUT output registers.

The assertions cover ready/valid ownership, held output payload under backpressure, complex-value and metadata transfer, strict threshold comparison, DC protection and the unique-bin eligibility rule. Equality must be retained. Covers exercise an occupied stall, simultaneous output consumption and input acceptance, protected DC below threshold, suppressed Nyquist, equality retention, just-below-threshold suppression and clear/recovery.

These properties relate the mask decision to the magnitude emitted by the DUT. They do not independently prove the multiplier, accumulated spectral statistics or the full FFT. The input thresh-

old is unconstrained per accepted transaction; whole-epoch threshold ownership remains an upstream contract checked by the system environment.

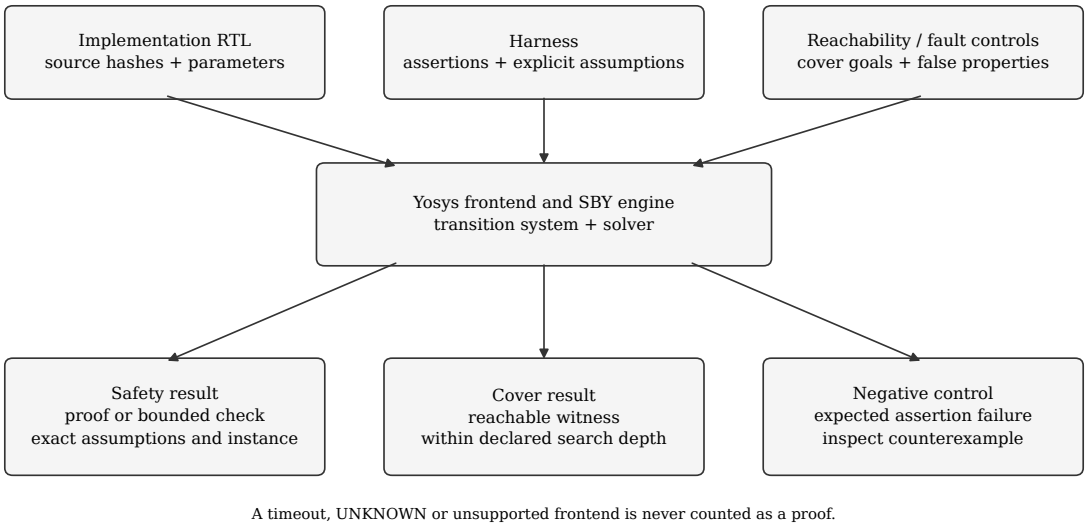


Figure 2: Formal flow and the scope of each result. Covers and deliberately failing controls are evaluated separately from safety proofs.

3.4 Result matrix

Table 4: Formal jobs grouped by their registered expected outcome.

Family	Proof	Cover	Expected failure
FIFO	4	2	1
Round/saturate	11	11	1
Magnitude mask	1	1	1
Total	16	14	3

The 30 ordinary jobs return solver PASS. The three negative jobs return solver FAIL, with a nonzero process exit, the intended assertion label and a counterexample VCD. The campaign accepts those failures as checker controls. Thus 33 accepted jobs does not mean 33 proved properties.

The 14 cover jobs reach 73 selected cover predicates. Multiple predicates may share one witness trace. Together with the three false-property controls, the campaign emits 68 native VCD files. The index preserves case IDs, engine settings, bounds and hashes. A cover establishes at least one trace within the configured depth, without proving a safety property or complete behavior coverage.

The pinned Linux tool suite is OSS CAD Suite 2026-10-16: Yosys, SBY, ABC, Z3 and the slang SystemVerilog frontend. The release archive and actual executables are identified by SHA-256. Sequential proofs use ABC PDR or SMT temporal induction; round/saturate checks use exhaustive symbolic combinational comparison. The production-depth FIFO proofs cover control only. The 256-entry PCM source uses different RTL and is not included in these FIFO proofs.

4 Debugging evidence and checker qualification

4.1 Failures retained and resolved

Early UVM harness attempts exposed unsupported multidimensional class-array file loading, start during memory initialization and a one-bit completion flag interpreted as signed decimal. These were verification-environment defects. The accepted harness loads compatible fixture arrays, waits for initialization and exports an integer completion flag. No DUT source changed. During the registered matrix, the simulator later terminated with a startup segmentation fault before DUT execution for seed 7. One explicit relaunch succeeded; the failed attempt, selected replacement and source/compile identities remain separate.

The first reduced FIFO induction attempt timed out. ABC PDR subsequently proved the same exact-order contract. Early mask attempts also exposed a frontend invocation error and an omitted package dependency. More substantially, internal symbolic registers were lowered to undefined port connections rather than shared symbolic inputs. This caused a spurious stall failure and an unreachable recovery cover. Explicit unconstrained top-level input ports repaired the harness/frontend interface. An independent review checked the lowered model, retained assertions/covers and unchanged reset assumption. The accepted mask proof did not obtain its result by restricting legal traffic or modifying the DUT.

4.2 Checker qualification

The UVM negative simulation mutates one expected waveform value. Acceptance requires the intended `Y_GOLD_MISMATCH` fatal message and nonzero simulator exit. The three formal negatives deliberately assert that a FIFO never fills, a rounding operation never saturates and the mask never produces valid output. Their labeled counterexamples exercise the corresponding tool/frontend failure paths. The reduced FIFO safety proof uses ABC PDR while its negative control uses SMT bounded checking.

Independent review also mutates the published package and recorded traces. Four UVM package mutations and ten observation/coverage mutations must be rejected, while both unmodified baselines pass. The formal publication checker separately accepts one unchanged baseline and rejects 12 mutations, including rehashed but inconsistent outcomes. These controls test admission, missing/extra records, numerical/order checks and receipt reconciliation. They are checker tests, not additional solver jobs, and do not establish detection of every possible defect.

4.3 Actual trace excerpts

The figure generator reads the committed clock-sample CSV and VCD, records their hashes, and exports the selected data alongside the figures. Conceptual architecture diagrams and measured trace excerpts are labeled separately.

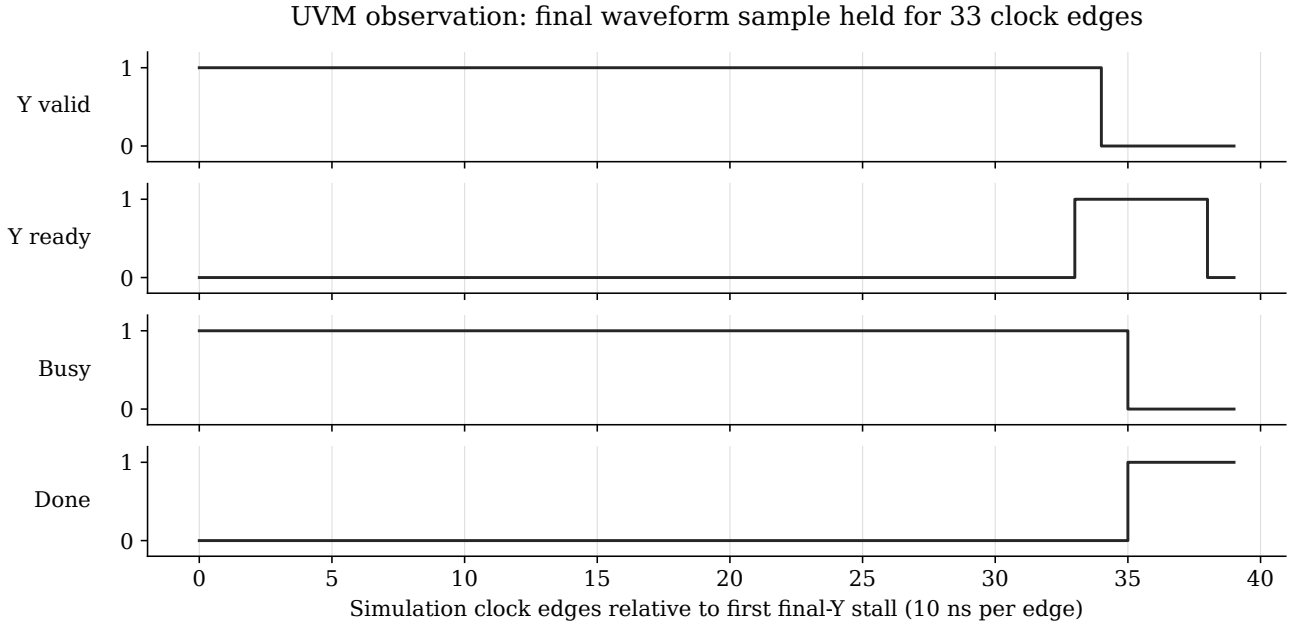


Figure 3: Actual seed-1 monitor samples around the final waveform token. Y remains valid while readiness is low for 33 edges. Busy remains asserted, then completion follows the accepted final transfer. The 10 ns simulation clock is a testbench setting, not a measured board-performance result.

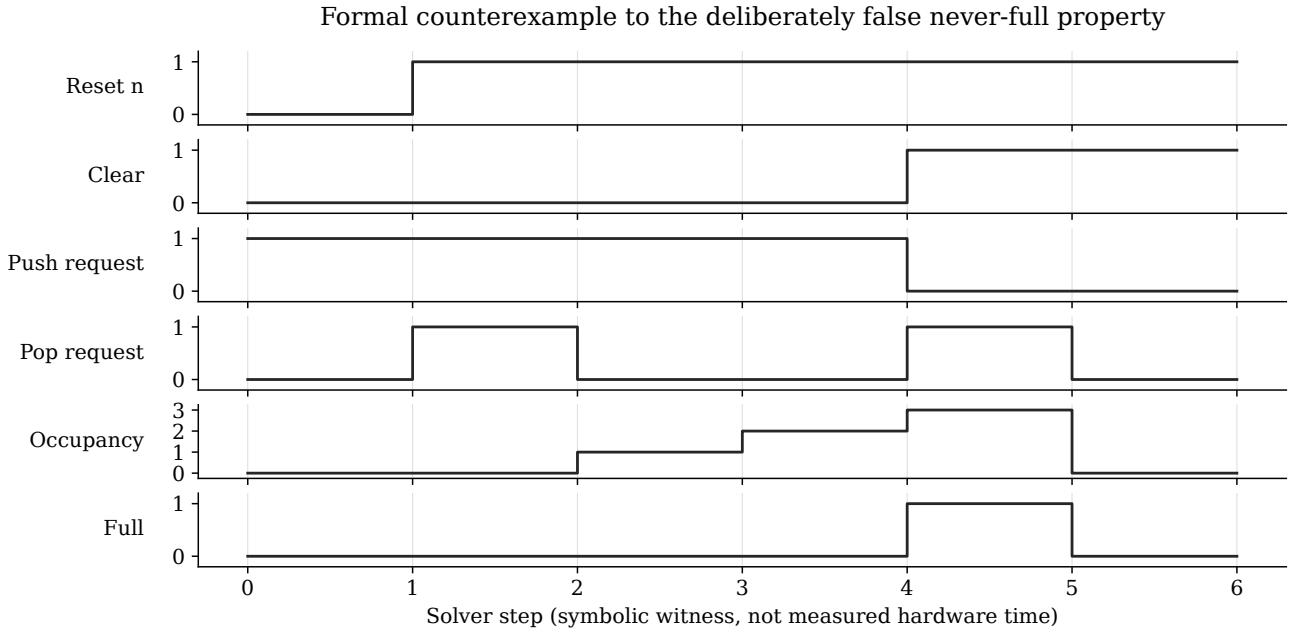


Figure 4: Native solver witness for the deliberately false FIFO never-full assertion. Occupancy reaches three in the DEPTH=3 instance, disproving the control property. Reset and CLEAR are displayed because they may change occupancy independently of pop requests. This is an expected negative control, not a reported DUT defect.

5 Traceability, reproduction and residual limits

The supplemental packages are `phase2/verification/uvm` and `phase2/verification/formal`. Their READMEs give the exact tool options, dependencies and evidence interpretation. The report package contains editable LaTeX, plotted trace data, figure generators and a byte-identity manifest. Hashes provide traceability; they are neither proof certificates nor a substitute for rerunning tools.

Package integrity checks use Python's standard library:

```
python phase2/verification/uvm/run.py --check-package
python phase2/verification/formal/run.py --repo-root phase2 --check-package
python phase2/docs/verification/uvm_formal/check_bundle.py
```

These commands do not execute the DUT or a solver. Actual UVM execution needs the admitted simulator and genuine UVM source library. The UVM runner uses a fresh output directory, invokes compile and simulation, applies deadlines and checks the numerical observations. Formal execution needs the separately downloaded pinned OSS CAD Suite:

```
python3 phase2/verification/formal/run.py \
  --repo-root phase2 --tool-bin /path/to/oss-cad-suite/bin \
  --output formal-runs/new-attempt
```

The full formal matrix is also configured as a dedicated GitHub Actions workflow. The existing Linux/Windows host workflow checks source hygiene, reference tests, RTL elaboration and package/evidence integrity. It does not silently substitute a package check for licensed UVM simulation.

Before publication, an independent review reconciles numerical counts, exact compiled inputs, proof assumptions, per-case outcomes and intended negative signatures. The original failed simulator and formal attempts remain distinguishable from accepted executions. Publishing a fresh commit does not make the historical board campaign a new hardware run.

5.1 Interpretation limits

No general FFT/IFFT equivalence, all-width arithmetic theorem, whole-system liveness, formal CDC/RDC, analog metastability, physical timing or process/voltage/temperature qualification is implied. Two-state formal analysis supplements the existing four-state simulation checks. The new simulation matrix does not repeat the full-length corpus or the board campaign. No power saving, laptop-CPU speed advantage, AI speedup or production reliability result follows from these verification additions.

References

[1] Accellera Systems Initiative, *Universal Verification Methodology Working Group*. `https://www.accellera.org/activities/working-groups/uvm`. Accessed October 17, 2026.

[2] YosysHQ, *SBY Documentation: Getting Started*. `https://symbiyosys.readthedocs.io/en/latest/quickstart.html`. Accessed October 17, 2026.

[3] YosysHQ, *OSS CAD Suite, release 2026-10-16*. `https://github.com/YosysHQ/oss-cad-suite-build/releases/tag/2026-10-16`.

[4] TRECAP project, *Registered TSH2 verification package and historical portfolio*. `https://github.com/DatDinhh/T_RECAP/tree/main/phase2/verification`.